

Product Blueprint & AI Constitution

Project: TikTok Live Monitor Dashboard

Purpose: This document replaces a traditional PRD. It is intended to become the single source of truth for product, design, engineering, and AI-assisted development (Taste Skill).

1. Product Philosophy

The application exists to provide a centralized desktop-first dashboard for monitoring many TikTok creators simultaneously. Users should not need to open TikTok repeatedly to determine who is currently live. The product prioritizes legality, stability, consistency, maintainability, and premium user experience over feature quantity.

2. Product Vision

Create a professional monitoring platform where users can browse creators, identify live sessions instantly, review analytics, and watch publicly available livestreams inside a unified interface whenever public playback is permitted. The system must never attempt to bypass authentication, DRM, or platform restrictions.

3. Core Principles

- Desktop-first.
- Dark professional SaaS UI.
- Single source of truth.
- Component reusability.
- Explicit decisions over assumptions.
- Every feature must have a defined purpose, success criteria, and edge-case handling.

4. AI Constitution

Taste Skill must always read this blueprint before generating code. It must never invent product behavior that is not documented. When requirements are ambiguous, it must request clarification instead of making assumptions. It must preserve naming, layout hierarchy, design tokens, folder structure, and reusable components.

5. Permanent Project Memory

Immutable project memory:

- Theme: Dark.
- Platform: Desktop-first.
- Sidebar is fixed and persistent.
- Main player remains centered.
- Live list always appears to the left of the player.
- Live Details remain on the right.
- Components are reused, never duplicated.
- Tailwind CSS + shadcn/ui only.
- TypeScript strict mode mandatory.

6. UI Blueprint

Dashboard consists of: Sidebar → Top Navigation → KPI Cards → Live List → Public Live Player → Live Details → Analytics → Recent Activity. No modal should interrupt the monitoring workflow. Context panels should be preferred over popup dialogs.

7. UX Rules

Every interaction must minimize clicks. Selecting a creator replaces the currently displayed livestream. Offline creators disable the Watch action and display 'Offline' rather than loading empty content. Unknown values are displayed as 'Unavailable' instead of zero.

8. Live Player Specification

The embedded player only renders livestreams that are publicly accessible. If public playback is unavailable, display an informative placeholder and an official action such as 'Open on TikTok'. Never attempt to circumvent authentication.

9. Functional Specification

Search, filtering, sorting, favorites, creator management, analytics, history, realtime updates, audit logs, workspace-ready architecture, settings, integrations, health monitoring, graceful loading states, retry strategy, and error banners.

10. Component Specification

Define Sidebar, KPI Card, Live Card, Player Container, Live Details Panel, Analytics Widgets, History Table, Search Bar, Status Badge, Empty State, Skeleton Loader, Error Banner, Confirmation Dialog, Admin Table, and Settings Form as independent reusable components with shared design tokens.

11. Technical Architecture

Frontend: Next.js 16. Backend: NestJS. Database: PostgreSQL (Supabase). Realtime: WebSocket/Supabase Realtime. State management should separate server state from UI state. API contracts must remain backward compatible.

12. Folder Structure Rules

AI must never create arbitrary folders. Features are organized by domain. Shared UI lives in a common components directory. Business logic must not be embedded inside presentation components.

13. Error Strategy

Every network request requires loading, success, empty, retry, and failure states. Three retry attempts maximum before presenting a recoverable error.

14. Security & Privacy

No TikTok credentials are stored. HTTPS only. Secrets remain server-side. Logs exclude personal or sensitive information. Least privilege access is mandatory.

15. Acceptance Criteria

Every feature must define measurable outcomes. UI implementation must visually match the approved desktop mockup. No unexplained deviation is acceptable.

16. Critical Thinking Checklist

Before implementing any change, Taste Skill must answer:

1. Why does this feature exist?
2. Does it conflict with existing memory?
3. Can an existing component be reused?
4. Does the change alter UX consistency?
5. Does it introduce technical debt?
6. Is there a documented acceptance criterion?

If any answer is unknown, implementation pauses pending clarification.

17. Future Documentation

This blueprint is Version 1. Future volumes should expand into complete BRD, UI Specification, Database Design, ERD, API Contracts, Sequence Diagrams, State Diagrams, Deployment Guide, Testing Strategy, Accessibility Guide, Coding Standards, and Design Tokens.